ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	1 семестр, 2025-2026 гг.

Практическое занятие №5

Подключение к PostgreSQL через `database/sql`. Выполнение простых запросов (`INSERT`, `SELECT`)

Цели:

1. Установить и настроить PostgreSQL локально.
2. Подключиться к БД из Go с помощью `database/sql` и драйвера PostgreSQL.
3. Выполнить параметризованные запросы `INSERT` и `SELECT`.
4. Корректно работать с `context`, пулом соединений и обработкой ошибок.

Теоретическая часть.

Подключение к PostgreSQL через `database/sql`

1. Реляционные базы данных и PostgreSQL

Реляционная база данных (РСУБД) хранит данные в виде таблиц, связанных между собой через ключи. PostgreSQL — это современная объектно-реляционная система управления базами данных (ОРСУБД), одна из самых популярных в промышленной разработке. Она поддерживает:

- строгую структуру данных (таблицы, схемы, типы данных),
- транзакции и целостность данных,
- SQL-стандарт (Structured Query Language).

Backend-разработчики используют PostgreSQL для хранения пользователей, заказов, логов, конфигураций и любых структурированных данных.

2. Подключение из Go: пакет `database/sql`

Go не имеет встроенной поддержки конкретной СУБД, но предоставляет универсальный интерфейс через пакет `database/sql`.

- `database/sql` работает как «слой абстракции»: разработчик использует одни и те же функции (`Query`, `Exec`, `Prepare`), а драйвер реализует конкретные вызовы к СУБД.
- Для PostgreSQL применяют драйвер `github.com/lib/pq` или более новый `github.com/jackc/pgx`.

Пример импорта:

```
import (
    "database/sql"
    _ "github.com/lib/pq"
)
```

Подключение:

```
dsn := "host=localhost port=5432 user=postgres password=secret dbname=mydb sslmode=disable"
db, err := sql.Open("postgres", dsn)
```

Здесь:

- `sql.Open` возвращает объект `*sql.DB` (это пул соединений, а не одно соединение);
- DSN (Data Source Name) содержит параметры подключения.

3. Основные операции SQL: `INSERT` и `SELECT`

Backend-приложения выполняют SQL-запросы для записи и чтения данных.

INSERT (создание записи):

```
INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com');
```

SELECT (чтение записей):

```
SELECT id, name, email FROM users;
```

В Go:

```
// вставка
_, err := db.Exec("INSERT INTO users (name, email) VALUES ($1, $2)", "Alice", "alice@example.com")

// выборка
rows, err := db.Query("SELECT id, name, email FROM users")
```

Важно: вместо конкатенации строк всегда использовать **плейсхолдеры** `$1, $2...` — это защищает от SQL-инъекций.

4. Работа с результатами запросов

- `db.QueryRow` возвращает одну строку, доступ к данным через `Scan`.
- `db.Query` используется для нескольких строк: нужно пройти по `rows.Next()` и сканировать каждую запись.

Пример:

```
row := db.QueryRow("SELECT id, name FROM users WHERE id=$1", 1)
var id int
var name string
```

```
    err := row.Scan(&id, &name)
```

5. Connection Pool (пул соединений)

Каждый вызов `sql.Open` не создает одно соединение, а управляет пулом. Это важно для производительности:

```
    db.SetMaxOpenConns(10)   // максимум активных соединений
    db.SetMaxIdleConns(5)    // соединений в простое
    db.SetConnMaxLifetime(time.Hour)
```

6. Ошибки и безопасность

При работе с PostgreSQL нужно учитывать:

- **SQL-инъекции** — использовать плейсхолдеры, а не строковую конкатенацию.
- **Ошибки подключения** — проверять `db.Ping()` при старте приложения.
- **Обработка ошибок** при `Scan`, `Exec` и `Query`.

7. Значение для backend-разработчика

Умение работать с PostgreSQL через `database/sql` — базовый навык.

- Почти все промышленные проекты используют реляционные базы данных.
- Даже если в будущем применяются ORM (например, GORM), понимание SQL и работы через `database/sql` необходимо.
- `INSERT` и `SELECT` — первые шаги к построению API, которые сохраняют и возвращают данные пользователям.

Практическая часть

Что устанавливаем заранее

- **PostgreSQL 14+**
  - Windows: установщик EnterpriseDB → задать пароль суперпользователя `postgres`.
  - macOS: `brew install postgresql && brew services start postgresql`
  - Linux (Debian/Ubuntu): `sudo apt install postgresql && sudo -u postgres psql`
- **Go 1.21+**: `go version`
- **psql** (идёт вместе с Postgres): `psql --version`

1) Создаём БД и таблицу

Откройте `psql`:

Windows: «SQL Shell (psql)» или PowerShell:

```
psql -U postgres
```

macOS/Linux:

```
psql -U postgres
```

Выполните:

```
-- создать БД (если ещё нет)
CREATE DATABASE todo;

-- подключиться к ней
\c todo

-- создать таблицу
CREATE TABLE IF NOT EXISTS tasks (
  id         SERIAL PRIMARY KEY,
  title      TEXT        NOT NULL,
  done       BOOLEAN     NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
);

-- быстрый тест
INSERT INTO tasks (title) VALUES ('Первая задача из psql');
SELECT * FROM tasks;
```

Ожидаемо увидите одну строку в `SELECT`.

2) Готовим Go-проект

Создайте папку и модуль:

```
mkdir pz5-db && cd pz5-db
go mod init example.com/pz5-db
go get github.com/jackc/pgx/v5/stdlib
```

Мы используем драйвер **pgx** в режиме совместимости с `database/sql`. Импортируем как `_ "github.com/jackc/pgx/v5/stdlib"`.

(Опционально для .env)

```
go get github.com/joho/godotenv
```

3) Структура файлов

Рекомендуем «плоско» для ПЗ:

```
pz5-db/
  main.go
  db.go
  repository.go
  go.mod
  .env            (по желанию)
```

3.1) `.env` (по желанию; удобно на занятии)

Создайте `.env` (в корне):

```
DATABASE_URL=postgres://postgres:YOUR_PASSWORD@localhost:5432/todo?sslmode=disable
```

Замените `YOUR_PASSWORD`.

3.2) `db.go` — подключение к БД и пул

```
package main

import (
  "context"
  "database/sql"
  "log"
  "time"

  _ "github.com/jackc/pgx/v5/stdlib"
)

func openDB(dsn string) (*sql.DB, error) {
  db, err := sql.Open("pgx", dsn)
  if err != nil {
    return nil, err
  }
  // настройки пула — достаточно для локалки
  db.SetMaxOpenConns(10)
  db.SetMaxIdleConns(5)
  db.SetConnMaxLifetime(30 * time.Minute)

  // проверка соединения с таймаутом
  ctx, cancel := context.WithTimeout(context.Background(),
2*time.Second)
  defer cancel()
  if err := db.PingContext(ctx); err != nil {
    return nil, err
  }
  log.Println("Connected to PostgreSQL")
  return db, nil
}
```

3.3) `repository.go` — минимальный репозиторий (INSERT, SELECT)

```
package main

import (
  "context"
  "database/sql"
  "time"
)

// Task — модель для сканирования результатов SELECT
type Task struct {
  ID        int
  Title     string
```

```
	Done	bool
	CreatedAt time.Time
}

type Repo struct {
	DB *sql.DB
}

func NewRepo(db *sql.DB) *Repo { return &Repo{DB: db} }

// CreateTask — параметризованный INSERT с возвратом id
func (r *Repo) CreateTask(ctx context.Context, title string)
(int, error) {
	var id int
	const q = `INSERT INTO tasks (title) VALUES ($1) RETURNING
id;`
	err := r.DB.QueryRowContext(ctx, q, title).Scan(&id)
	return id, err
}

// ListTasks — базовый SELECT всех задач (демо для занятия)
func (r *Repo) ListTasks(ctx context.Context) ([]Task, error)
{
	const q = `SELECT id, title, done, created_at FROM tasks
ORDER BY id;`
	rows, err := r.DB.QueryContext(ctx, q)
	if err != nil {
		return nil, err
	}
	defer rows.Close()

	var out []Task
	for rows.Next() {
		var t Task
		if	err	:=	rows.Scan(&t.ID,	&t.Title,	&t.Done,
&t.CreatedAt); err != nil {
			return nil, err
		}
		out = append(out, t)
	}
	return out, rows.Err()
}
```

3.4) `main.go` — запуск: читаем DSN → вставляем → читаем

```
package main

import (
```

```
      "context"
      "fmt"
      "log"
      "os"
      "time"

      "github.com/joho/godotenv"
     )

     func main() {
      // .env не обязателен; если файла нет — ошибка игнорируется
      _ = godotenv.Load()

      dsn := os.Getenv("DATABASE_URL")
      if dsn == "" {
       // fallback — прямой DSN в коде (только для учебного
стенда!)
        dsn                                                    =
"postgres://postgres:YOUR_PASSWORD@localhost:5432/todo?sslmode=d
isable"
      }

      db, err := openDB(dsn)
      if err != nil {
       log.Fatalf("openDB error: %v", err)
      }
      defer db.Close()

      repo := NewRepo(db)

      // 1) Вставим пару задач
      ctx,  cancel  :=  context.WithTimeout(context.Background(),
3*time.Second)
      defer cancel()

      titles  :=  []string{"Сделать  ПЗ  №5",  "Купить  кофе",
"Проверить отчёты"}
      for _, title := range titles {
       id, err := repo.CreateTask(ctx, title)
       if err != nil {
            log.Fatalf("CreateTask error: %v", err)
       }
       log.Printf("Inserted task id=%d (%s)", id, title)
      }

      // 2) Прочитаем список задач
```

```
      ctxList,                    cancelList                    :=
context.WithTimeout(context.Background(), 3*time.Second)
      defer cancelList()

      tasks, err := repo.ListTasks(ctxList)
      if err != nil {
        log.Fatalf("ListTasks error: %v", err)
      }

      // 3) Напечатаем
      fmt.Println("=== Tasks ===")
      for _, t := range tasks {
        fmt.Printf("#%d | %-24s | done=%-5v | %s\n",
            t.ID,                t.Title,               t.Done,
t.CreatedAt.Format(time.RFC3339))
      }
    }
```

В учебных целях допустим прямой DSN в коде. В реальных проектах всегда используем переменные окружения/секрет-хранилища.

4) Запуск и проверка

Из корня проекта:

```
go run .
```

Ожидаемый лог:

```
Connected to PostgreSQL
Inserted task id=2 (Сделать ПЗ №5)
Inserted task id=3 (Купить кофе)
Inserted task id=4 (Проверить отчёты)
=== Tasks ===
#1 | Первая задача из psql     | done=false | 2025-09-22T...
#2 | Сделать ПЗ №5             | done=false | 2025-09-22T...
#3 | Купить кофе               | done=false | 2025-09-22T...
#4 | Проверить отчёты          | done=false | 2025-09-22T...
```

(Номера могут отличаться — зависит от того, создавали ли вы строки ранее через `psql`.)

Проверим через `psql`:

```
SELECT id, title, done, created_at FROM tasks ORDER BY id;
```

Проверочные задания (в этой же работе)

1. Реализуйте функцию:

```
func (r *Repo) ListDone(ctx context.Context, done bool) ([]Task, error)
```

которая вернёт только выполненные (`done=true`) или невыполненные (`done=false`) задачи.

Подсказка: `WHERE done = $1`.

2. Добавьте функцию:

```
func (r *Repo) FindByID(ctx context.Context, id int) (*Task, error)
```

и выведите в `main` подробности по задаче с конкретным `id`.

3. Сделайте функцию массовой вставки `CreateMany(ctx, titles []string) error` через транзакцию.

4. Измерьте и объясните выбор настроек пула (`SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxLifetime`) для вашей машины.

Требования к отчёту

1. **В Git** (ПЗ №5, ФИО, группа, дата, тема).
2. Описание окружения: версии Go/PostgreSQL, ОС.
3. Скриншоты:
   - создание БД/таблицы в `psql`;
   - успешный вывод `go run .` (вставка и список задач);
   - `SELECT * FROM tasks;` в `psql`.
4. Код: `db.go`, `repository.go`, фрагменты `main.go`.
5. Краткие ответы:
   - Что такое пул соединений `*sql.DB` и зачем его настраивать?
   - Почему используем плейсхолдеры `$1, $2`?
   - Чем `Query`, `QueryRow` и `Exec` отличаются?
6. Обоснование транзакций и настроек пула.

Частые ошибки и быстрые решения

- **`pq: password authentication failed` / `invalid password`** — проверьте логин/пароль и DSN; убедитесь, что Postgres запущен.
- **`connect: connection refused`** — сервер не запущен/порт не 5432.
- **`relation "tasks" does not exist`** — вы создали таблицу не в той БД; сначала `\c todo`, затем `CREATE TABLE`.
- **Зависание запроса** — используйте `context.WithTimeout`.
- **`Scan` паникует/ругается** — типы должны соответствовать колонкам (`bool` ↔ `BOOLEAN`, `time.Time` ↔ `TIMESTAMPTZ`).
- **Русский текст «кракозябрами» в PowerShell** — выполните `chcp 65001` перед запуском `curl`/команд.

Мини-чек-лист перед сдачей

- [ ] `go run .` подключается к БД и пишет «Connected to PostgreSQL».
- [ ] После запуска в таблице есть новые строки (лог `Inserted task id=...`).
- [ ] Консоль выводит список задач (как минимум 3).
- [ ] В `psql` `SELECT * FROM tasks;` показывает те же записи.
- [ ] В коде **везде** используются плейсхолдеры `$1, $2, ...` (никакой конкатенации строк SQL!).
- [ ] Настроен `context.WithTimeout` в местах запросов.